

day09 课堂笔记

0.复习和反馈

老师能不能给透露一下考试的大题，在哪个方向

1. 继承的基本语法[掌握]

1 继承：描述的类与类之间所属关系。

2 基本语法：

3 `class 类B(类A):`

4 `pass`

5

6 称为类 B 继承类 A

7 特点：B类的对象可以使用 A类的属性和方法

8 优点：代码复用.重复相同的代码不用多次书写。

9 名词：

10 类A：父类 基类

11 类B：子类 派生类

```

1  # 1. 定义是个 动物类 animal类
2  class Animal(object):
3      # 2. 在animal类书写 play方法, 输出快乐的玩耍....
4      def play(self):
5          print('快乐的玩耍....')
6
7
8  # 3. 定义Dog类继承animal类,
9  class Dog(Animal):
10     pass
11
12
13 # 4. 创建dog类对象. 调用父类的方法
14 dog = Dog()
15 dog.play()

```

1 继承：描述的类与类之间所属关系。

2 基本语法：

```

3  class 类B(类A):
4      pass

```

6 称为类 B 继承类 A

7 特点：B类的对象可以使用 A类的属性和方法

8 优点：代码复用. 重复相同的代码不用多次书写。

9 名词：

10 类A：父类	基类
11 类B：子类	派生类

Run: 01-继承的基本语法 x

C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day09/01-继承的基本语法.py
快乐的玩耍....

Process finished with exit code 0

2. 单继承和多层继承[理解]

```

8  # 1. 定义是个 动物类 animal类
9  class Animal(object): # 对于Animal类和object类来说, 单继承
10     # 2. 在animal类书写 play方法, 输出快乐的玩耍....
11     def play(self):
12         print('快乐的玩耍....')
13
14
15 # 3. 定义Dog类继承animal类,
16 class Dog(Animal): # Dog --> Animal 也是单继承, Dog --> Animal --> object 这种继承关系称为多层继承
17     def bark(self):
18         print('汪汪汪叫.....')
19
20
21 # 定义类 XTQ类, 继承 Dog类
22 # 多层继承中, 子类可以使用所有继承链中的类中的方法和属性
23 class XTQ(Dog): # XTQ --> Dog 单继承, XTQ --> Dog --> Animal 类, 多层继承
24     pass

```

```

1  """
2  单继承: 如果一个类只有一个父类, 把这种继承关系称为单继承
3  多继承: 如果一个类有多个父类, 把这种继承关系称为多继承
4  多层继承: C --> B --> A |
5  """

```

Run: 02-继承的分类 x

C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day09/02-继承的分类.py
汪汪汪叫.....
快乐的玩耍....

3. 子类重写父类的同名方法[掌握]

- 1 重写：子类定义和父类名字相同的方法。
- 2 为什么重写：父类中的方法，不能满足子类对象的需求，所以要重写。
- 3 重写之后的特点：子类对象调用子类自己的方法，不再调用的方法，父类对象调用父类自己的方法。

```
1 # 1. 定义Dog类, 书写bark方法, 输出 汪汪汪叫
2 class Dog(object):
3     def bark(self):
4         print('汪汪汪叫.....')
5
6
7 # 2. 定义XTQ类, 继承Dog类. 重写父类中的bark方法, 输出 嗷嗷嗷叫
8 class XTQ(Dog):
9     def bark(self):
10        print('嗷嗷嗷叫-----')
11
12
13 # 创建Dog类对象
14 dog = Dog()
15 dog.bark() # 父类自己的
16
17 # 创建XTQ类对象
18 xtq = XTQ()
19 xtq.bark()
20
```

- 1 重写：子类定义和父类名字相同的方法。
- 2 为什么重写：父类中的方法，不能满足子类对象的需求，所以要重写。
- 3 重写之后的特点：子类对象调用子类自己的方法，不再调用的方法，父类对象调用父类自己的方法。

```
C:\Develop\python38\python.exe C:/
汪汪汪叫.....
嗷嗷嗷叫-----
Process finished with exit code 0
```

4. 子类调用父类的同名方法[掌握]

```

8 class XTQ(Dog):
9     def bark(self):
10         print('嗷嗷嗷叫-----')
11
12     def see_host(self):
13         """看见主人之后,要汪汪汪叫,不能嗷嗷嗷叫"""
14         print('看见主人了,', end='')
15         # self.bark()
16         # 想要在子类中调用父类的同名方法
17         # 方法一: 父类名.方法名(self, 其他参数) 通过实例对象.方法名() 调用方法,不需要给self传递实参值,
18         # python解释器会自动将对象作为实参值传递给self形参,如果是通过类名.方法() 调用,python解释器就
19         # 不会自动传递实参值,需要手动给self形参传递实参值
20         Dog.bark(self)
21
22         # 方法二 super(类A, self).方法名(参数), 会调用当类A 的父类中的方法
23         super(XTQ, self).bark() # 调用 XTQ类父类中的bark方法
24
25         # 方法三 是方法二的简写, super().方法名(参数) ==> super(当前类, self).方法名()
26         super().bark()
27
28         # 创建XTQ类对象
29         xtq = XTQ()
30         xtq.see_host()
31

```

注意事项: 需要手动的传递 self 形参

看见主人了,汪汪汪叫.....
汪汪汪叫.....
汪汪汪叫.....

激活 Windows

5. 继承中的 init [掌握]

```

1 # 1. 定义Dog类
2 class Dog(object):
3     def __init__(self, name):
4         # 添加属性
5         self.age = 0
6         self.name = name
7
8     def __str__(self):
9         return f'名字为:{self.name}, 年龄为{self.age}'
10
11     # 如果子类重写了父类的 init 方法, 在子类中需要调用父类的 init 方法, 给对象添加从父类继承的属性
12 # 2. 定义XTQ类继承Dog类
13 class XTQ(Dog):
14     # 子类重写了父类的__init__ 方法, 默认不再调用父类的init方法, 需要手动的调用父类的init方法
15     def __init__(self, name, color):
16         # 注意: 子类 init 方法的形参, 一般都先写父类的形参, 再写自己自己独有的形参
17         super().__init__(name)
18         self.color = color
19
20     def __str__(self):
21         return f'名字为:{self.name}, 年龄为{self.age}, 毛色为:{self.color}'
22
23 # 3. 创建XTQ类对象
24 xtq = XTQ('小黑', '红色')
25 print(xtq)
26

```

名字为:小黑, 年龄为0, 毛色为:红色

激活 Windows

6. 多继承[理解]

如果一个类有两个及以上的父类,就把这种继承关系称为多继承.

```
10 # 2. 定义God类, 定义 play方法和eat方法
11 class God(object):
12     def play(self):
13         print('在云中飘一会....')
14
15     def eat(self):
16         print('吃蟠桃仙丹....')
17
18
19 # 3. 定义XTQ类, 继承Dog类和God类
20 # class XTQ(Dog, God): # XTQ 类有两个父类, 这个继承关系称为多继承, XTQ类对象, 可以调用两个父类中的属性和方法
21 class XTQ(God, Dog): # XTQ 类有两个父类, 这个继承关系称为多继承, XTQ类对象, 可以调用两个父类中的属性和方法
22     pass
23
24
25 # 4. 创建XTQ类对象
26 xtq = XTQ()
27 xtq.bark() # 调用 Dog父类中的方法
28 xtq.play() # 调用 God父类中的方法
29
30 xtq.eat() # 两个父类都存在eat方法, 子类对象调用的是 第一个父类中的方法
31
```

```
21 class XTQ(God, Dog): # XTQ 类有两个父类, 这个继承关系称为多继承, XTQ类对象, 可以调用两个父类中的属性和方法
22     def eat(self):
23         print('子类重写eat方法, 调用子类自己的方法')
24         # 调用指定父类中的方法
25         # 方法一 类名.方法名(self, 参数)
26         # Dog.eat(self)
27         # God.eat(self)
28
29         # 方法二 super(类A, self).方法名(参数) 类A的父类(继承顺序链的下一个类)中的方法
30         # super(XTQ, self).eat() # God 类中的方法
31         super(God, self).eat() # 调用的Dog类中的方法
32         # super(Dog, self).eat() # 调用的object类中的方法, 注意: object类中没有eat方法, 代码报错
33
34 (<class '__main__.XTQ'>, <class '__main__.God'>, <class '__main__.Dog'>, <class 'object'>)
35
36 # 4. 创建XTQ类对象
37 xtq = XTQ()
38 xtq.bark() # 调用 Dog父类中的方法
39 xtq.play() # 调用 God父类中的方法
40
41 xtq.eat() # 两个父类都存在eat方法, 子类对象调用的是 第一个父类中的方法
42
43 # 类名.__mro__ 可以当前类的继承顺序链, 也叫做方法的调用顺序
44 # print(XTQ.__mro__)
```

7. 私有权限[理解]

- 1 访问权限控制：在什么地方可以使用和操作。
- 2 私有权限：
- 3 定义：在方法和属性前加上两个下划线，就变为私有。
- 4 1. 不能在类外部通过对象直接访问和使用，只能在类内部访问和使用
- 5 2. 不能被子类继承，
- 6 公有：不是私有的，就是公有的。

```
1 """
2 私有属性，只需要在原属性名前加上两个下划线，即可
3 目的：保证数据的相对安全，
4 想要访问和使用私有属性：定义一个公有的方法，通过这个方法使用
5 """
6
7
8 # 案例需求：定义People类，定义属性 ICBC_money，钱不能随便被修改，必须是合法的终端才可以操作
9 class People(object):
10     def __init__(self):
11         # python中的私有本质是 修改属性的名字，在创建对象的时候，会自动的修改属性名
12         # 在属性名的前边加上 类名前缀
13         self.__ICBC_money = 0 # 定义私有属性
14
15     # 定义公有的方法，提供接口，修改余额
16     def get_money(self):
17         return self.__ICBC_money
18
19     def set_money(self, money):
20         self.__ICBC_money += money
21
22 # 创建People类对象
```

赋值之前: {'_People__ICBC_money': 0}

赋值之后: {'_People__ICBC_money': 0, '__ICBC_money': 1000}

1000

=====

0

1000

500

```
22 # 创建People类对象
23 xw = People()
24 # 实例对象.__dict__ 可以查看对象具有的属性信息，类型是字典，字典的key是属性名，字典的value是属性值
25 print('赋值之前:', xw.__dict__)
26 # print(xw.__ICBC_money)
27 xw.__ICBC_money = 1000 # 不是修改私有属性，是重新添加一个公有属性
28 print('赋值之后:', xw.__dict__)
29 print(xw.__ICBC_money)
30 print('=' * 20)
31 print(xw.get_money()) # 0
32 xw.set_money(1000)
33 print(xw.get_money()) # 1000
34 xw.set_money(-500)
35 print(xw.get_money()) # 500
```

```
1 """
2 私有方法：在方法的前边加上两个__，就为私有方法
3 私有方法，不能在类外部访问
4 作用：一般作为类内部的方法使用，不让在外部直接调用，保证业务逻辑不被破坏
5 """
6
7
8 class Dog(object):
9     def born(self):
10         """生小狗的方法，生一个小狗，休息30天"""
11         print('生了一只小狗...')
12         self.__sleep()
13
14     def __sleep(self):
15         print('休息30天')
16
17
18 dog = Dog()
19 # dog.__sleep()
20 dog.born()
21
```

8. 类属性[理解]

```
16
17
18 class Dog(object):
19     # 定义类属性, 类名
20     class_name = '狗类'
21
22     def __init__(self, name, age):
23         # 定义的都是实例属性
24         self.name = name
25         self.age = age
26
27
28 # 创建Dog 类对象
29 dog = Dog('大黄', 2)
30 print(dog.__dict__) # 打印dog对象具有的属性
31
32 # 类名.__dict__ 查看类对象具有的属性
33 print(Dog.__dict__)
34
35 {'name': '大黄', 'age': 2}
36 {'__module__': '__main__', 'class_name': '狗类', '__init__': <function Dog.__init__ at 0x000001FA179A5F70>}
```

Process finished with exit code 0

对象(实例对象): 通过class定义类创建, 即通过类实例化来的, 又称为实例, 实例对象实例对象定义的属性称为实例属性. 通过实例对象(self) 定义的属性都是实例属性. 实例属性: 每个实例对象中都存在一份, 并且值可能是不一样的

类(类对象): 通过class定义的, 又称为类对象, 是python解释器在创建类的时候自动创建的作用: 1. 通过类对象, 去定义实例对象; 2. 类对象可以保存一些属性信息, 称为类属性. 类属性的定义: 在类内部, 方法外部定义的变量就是类属性. 类属性, 字内存中只有一份

如何确定一个属性是该定义为实例属性还是类属性?
先假设这个属性为实例属性, 查看这个属性值对于不同的实例对象, 属性值是否都一样, 并且需要同时变化. 如果是, 则可以定义为类属性. 如果不是, 则可以定义为实例属性

```
35 # 访问类属性
36 # 类名.类属性
37 print(Dog.class_name)
38
39 # 修改类属性 类名.类属性 = 属性值
40 Dog.class_name = 'Dog类'
41
42 print(Dog.class_name)
43
44 # 补充, 注意: 如果不存在和实例属性名相同的类属性, 则可以使用实例对象访问类属性的值
45 # 如果存在重名, 则使用实例属性访问的一定是实例属性, 不是类属性
46 print(dog.class_name)
```

狗类
Dog类
Dog类

Process finished with exit code 0

9. 类方法

```

11 class Dog(object):
12     class_name = '狗类'
13
14     def __init__(self, name, age):
15         self.name = name
16         self.age = age
17
18     def play(self): # 实例方法
19         print(f"小狗{self.name} 在快乐的玩耍....")
20
21     # def get_class_name(self): # 是实例方法, 因为没有使用实例属性, 所以可以定义为类方法
22     #     return Dog.class_name
23
24     @classmethod
25     def get_class_name(cls): # cls 是类方法的默认形参, 在调用的时候, 不需要手动传递, python解释器会自动传递
26         return cls.class_name
27
28
29 dog = Dog('大黄', 2)
30 dog.play()
31 print(dog.get_class_name()) # 对象.类方法()
32
33 # 类名.类方法()
34 print(Dog.get_class_name())

```

1 """
2 实例方法: 类中默认定义的方法, 就是实例方法, 第一个参数为self, 表示实例对象
3
4 类方法: 使用 @classmethod 装饰的方法, 称为类方法, 第一个参数是cls, 代表的是类对象自己
5 什么情况定义为实例方法, 什么情况定义为类方法?
6 1. 如果在方法中使用了实例属性, 那么该方法必须定义为实例方法
7 2. 前提: 不需要使用实例属性. 需要使用类属性, 可以将这个方法定义为类方法
8 """

激活 Windows

10. 静态方法[掌握]

```

12 class Dog(object):
13     class_name = '狗类'
14
15     def __init__(self, name, age):
16         self.name = name
17         self.age = age
18
19     def play(self): # 实例方法
20         print(f"小狗{self.name} 在快乐的玩耍....")
21
22     @staticmethod # 定义静态方法
23     def show_info(): 静态方法如果有参数, 就必须传递实参值
24         print('这是一个Dog类')
25
26
27 dog = Dog('大黄', 2)
28 dog.play()
29 # 对象.方法名()
30 dog.show_info()
31 # 类名.方法名()
32 Dog.show_info()
33

```

1 """
2 实例方法: 类中默认定义的方法, 就是实例方法, 第一个参数为self, 表示实例对象
3 类方法: 使用 @classmethod 装饰的方法, 称为类方法, 第一个参数是cls, 代表的是类对象自己
4 静态方法: 使用 @staticmethod 装饰的方法, 称为静态方法, 对参数没有特殊要求, 可以有, 可以没有
5 什么情况定义为实例方法, 什么情况定义为类方法, 什么情况下静态方法?
6 1. 如果在方法中使用了实例属性, 那么该方法必须定义为实例方法
7 2. 前提: 不需要使用实例属性. 需要使用类属性, 可以将这个方法定义为类方法
8 3. 前提: 不需要使用实例属性, 同时也不需要类属性, 此时可以将这个方法定义为静态方法
9 """

11. 多态[理解]

1 """
2 在需要使用父类对象的地方,也可以传入子类对象得到不同的结果 ---- 多态
3 实现步骤:
4 1. 子类继承父类
5 2. 子类重写父类中的同名方法
6 3. 定义一个共同的方法, 参数为父类对象, 在方法中调用子类和父类同名的方法
7 """

10 # 1. 定义Dog类
11 class Dog(object):
12 def __init__(self, name):
13 self.name = name
14
15 def play(self):
16 print(f'小狗{self.name} 在玩耍.....')
17
18
19 # 2. 定义哮天犬类, 继承Dog类
20 class XTQ(Dog):
21 # 3. 重写 play方法
22 def play(self):
23 print(f'{self.name} 在天上追云彩.....')
24
25
26 # 4. 定义一个共同的方法,
27 def play_with_dog(obj_dog):
28 obj_dog.play()
29

31 # 创建Dog类对象@
32 dog = Dog('大黄')
33 play_with_dog(dog)
34
35 # 创建一个XTQ类的对象
36 xtq = XTQ('小黑')
37 play_with_dog(xtq)
38

这个代码可以写很多, 这里为了简写
C:\develop\python3\python.exe C:/...
小狗大黄 在玩耍.....
小黑 在天上追云彩.....
Process finished with exit code 0

13-
法

13-
1.8

1 鸭子类型